

Advanced Hierarchical RAG Retrieval & Clustering Engine

Author: Core Engineering Team

Date: September 29, 2026

Classification: Proprietary System Architecture

1. The Core Problem: Why Do We Need Clustering in RAG?

In a traditional, flat Retrieval-Augmented Generation (RAG) system, user questions are directly mapped into a vector space and compared against thousands or millions of individual document chunks simultaneously. This approach has critical limitations that an interviewer will probe for. Our system completely resolves these via **Top-Down Tree Filtering** using hierarchical clustering.

The Problems Solved by Clustering:

- **Elimination of Semantic Noise:** In a database with thousands of chunks, unrelated topics frequently share similar phrasing or keywords. A flat search lets these irrelevant text chunks compete blindly, leading to *false positives* in the top-K results.
- **Prevention of Context Dilution:** If noisy, semi-relevant chunks fill the LLM's context window, the actual ground-truth answer gets drowned out. This prevents the LLM from hallucinating or losing the target instruction.
- **Computation Efficiency (Cross-Encoder Optimization):** Cross-encoders provide incredibly high-precision reranking, but they are computationally expensive ($O(N)$ runtime scaling heavily with text length). By using clustering to filter out 95% of the database before the cross-encoder fires, we stop it from wasting precious GPU/MPS compute cycles on useless data.
- **Macro-to-Micro Semantic Routing:** Clustering acts like a high-level **Domain Name System (DNS) Router**. It groups similar records together mathematically, allowing the system to locate the precise topic domain within milliseconds.

Component / Dimension	Standard Flat RAG	Our Clustered RAG Engine
Search Space	Entire Vector DB (100% chunks compete)	Restricted Topic Cluster (~5% search space)
Noise & Hallucinations	High risk due to out-of-context matches	Extremely low; strict domain isolation
Cross-Encoder Load	Heavy; evaluates massive candidate pools	Light; only runs on verified top candidates

2. Ingestion & Offline Clustering Service

The background ingestion service pipeline processes raw documents into an enriched, clustered graph matrix structure. It runs in two continuous pipelines: **Bulk Ingest** and the **Clustering Job Loop**.

Bulk Ingestion Pipeline Flow:

1. Data enters the system and triggers an orchestration workflow, populating chunks into dsprawl.chunks.
2. Chunks are embedded natively using the SentenceTransformer-BAAI/bge-small-en-v1.5 model (producing a 384-dimensional vector space) and stored in dsprawl.embedded_chunks.
3. The vectors and texts are indexed inside the primary Elasticsearch engine under the es_documents production index.

The Clustering Job (Step-by-Step Execution):

Step 1: Initialization (cluster_job.py & vector_loader.py)

When Phase 4 calls run_clustering_job(), it initializes a timestamped temporary index (e.g., clusters_v3_temp_20260929). It boots vector_loader.py to execute an Elasticsearch scroll query against es_documents. It downloads all raw chunks, extracts their string chunk_ids, and parses their raw vector arrays into a single, contiguous 2D NumPy float matrix (size: N × 384) using np.vstack(vectors).

Step 2: Level-1 Macro Boxing (agglomerative.py)

The matrix is passed directly to run_agglomerative_clustering() to perform bottom-up hierarchical clustering. Because it uses metric='cosine', it group chunks based on semantic alignment angle instead of raw spatial distance. Scikit-learn's fit_predict() outputs a 1D integer array mapping positions to cluster labels. For example, returning {'lvl1-f8d8c': [0, 14, 22]} means rows 0, 14, and 22 in the matrix belong to that exact root cluster. cluster_builder.py maps these matrix positions back into real string member_ids using its parallel string index lookup.

Step 3: Level-2 Micro Partitioning (kmeans_subclusters.py)

For every macro-cluster identified by Agglomerative Clustering, cluster_builder.py extracts the matching sub-matrix slice and shifts execution to K-Means. K-Means uses an adaptive policy (choose_k_balanced) to auto-split large clusters into smaller, highly cohesive sub-topics (prefixed with SC_...). It records the **Centroid Vector** for each group. If K=1, it takes the arithmetic average via np.mean(vectors, axis=0); if K>1, it pulls the

calculated centers from Scikit-learn directly.

Step 4: The Summarizer & Tag Injection Relay (tag_enrichment.py)

For each sub-cluster, kmeans_subclusters.py passes the member text payloads to an LLM via summarize_cluster(). The LLM creates a concise 2-sentence summary, which is embedded back into a 384-dimensional vector. tag_enrichment.py reads these metadata summaries and overwrites/appends them as dynamic tags inside the production index.

3. The Online Retrieval & Synthesis Pipeline

When a user query enters the system, the application abandons broad, flat vector matching. Instead, it utilizes the **Top-Down Tree Filtering** strategy generated by the clustering job, routing the query through clusters_v3 to perform high-precision isolation before matching text.

Detailed Execution Trace of a Query:

Step 1: Question Embedding

The incoming query string (e.g., 'What are the ATM transaction limits for SBI card holders?') is converted into a standard 384-dimensional vector (qvec) utilizing the native BAAI/bge-small-en-v1.5 pipeline.

Step 2: Root Cluster Selection

The application layer executes a high-speed vector search (k=1 or k=2) strictly against documents in the clusters_v3 index filtered where level: 1. It compares qvec against the macro-centroids. In this trace, it matches lv1-f8d8cdcb102b (Broad Banking Rules macro-cluster) because its centroid vector exhibits the closest cosine angle to the user's intent.

Step 3: Sub-cluster Traversal

The code opens the winning root cluster's payload to parse its list of child subclusters:

```
"subclusters": ["SC_lv1-f8d8cdcb102b_01", "SC_lv1-f8d8cdcb102b_02", ...]
```

It performs a secondary vector search limited strictly to those child sub-cluster IDs, matching qvec against their precise micro-group centroids. The micro-cluster SC_lv1-f8d8cdcb102b_02 (ATM Rules micro-group) scores the highest.

Step 4: Collecting Core Chunk IDs & Fallbacks

The application reads the document payload for that winning micro-cluster from clusters_v3 and extracts the literal chunk_ids string array:

```
"chunk_ids": ["chunk_uuid_111", "chunk_uuid_222", "chunk_uuid_333", ...]
```

Fallback Rule: If a root cluster had no subclusters due to small member size, the routing logic automatically skips sub-cluster traversal and reads the root cluster's own chunk_ids payload directly.

Step 5: High-Precision Restricted Retrieval (es_documents)

The isolated chunk_ids array is passed directly into the ChunkRetriever module via the restrict_ids parameter. The _ann method translates this list into a hard Elasticsearch boolean filter clause:

```
# Elasticsearch Boolean Filter Injection
"filter": [{"terms": {"chunk_id.keyword": restrict_ids}}]
```

Crucially, both the Approximate Nearest Neighbor (ANN) vector search and the traditional keyword-based BM25 search are bound by this hard constraint filter. The remaining 95%+ of chunks in the database are entirely ignored during evaluation.

Step 6: Blending, Reranking & Synthesis

1. **Reciprocal Rank Fusion (RRF):** Blends the restricted vector similarity and BM25 scores into a balanced, uniformly scaled candidate list.
2. **Cross-Encoder Validation:** The top candidate chunk records are passed directly into the Cross-Encoder model running on the hardware acceleration layer (device='mps') to score true syntax and semantic alignment.
3. **LLM Generation:** The final top 3 ranked context chunks are sent to the generation LLM, delivering an

answering window completely free of background noise and context dilution.